

Informe de Seguridad — OWASP Top 10 (2021)

Sistema MediClick — Plataforma de gestión clínica multi-tenant

Proyecto	MediClick (NestJS + GraphQL + Prisma + PostgreSQL / Next.js)
Alcance	Auditoría y remediación dirigida sobre las 10 categorías OWASP, backend server/
Rama de trabajo (ronda 1)	feat/payments-mp-redirect
Integrado a main (ronda 1)	PR #35 — merge commit e754494 (2026-06-30)
Ronda 2 — cierre de pendientes	rama feat/interop-fase1-proyeccion , commits 882474d y cc7b24c (2026-07-09)
Fecha del informe	30 de junio de 2026 — actualizado 9 de julio de 2026

1. Resumen ejecutivo

Se realizó una auditoría dirigida del proyecto MediClick contra las 10 categorías del **OWASP Top 10 (edición 2021)**. De las categorías evaluadas, se identificaron y corrigieron hallazgos concretos en **A01 (Control de acceso roto)**, **A02 (Fallas criptográficas)**, **A06 (Componentes vulnerables)** y **A09 (Fallas de registro y monitoreo)**. El hallazgo más severo —ausencia de aislamiento multi-tenant en la API GraphQL, la única API autenticada del sistema— fue verificado a nivel de código y en tiempo de ejecución, y corregido con prueba de regresión.

El 9 de julio de 2026 se cerraron los dos pendientes puntuales que había dejado abiertos la ronda 1: el saneamiento de las dependencias directas vulnerables (A06) y el endpoint de consulta de la bitácora de seguridad (A09). El resto de las categorías (A03, A04, A05, A07, A08, A10) sigue sin auditoría dirigida; se documentan al final con las mitigaciones estructurales existentes y lo pendiente de revisar.

Categoría	Nombre	Estado	Resultado
A01	Control de acceso roto (Broken Access Control)	CORREGIDO	Tenant scoping GraphQL + guard defense-in-depth en updateClinic + auditoría read-before-write
A02	Fallas criptográficas (Cryptographic Failures)	CORREGIDO	Cost factor de bcrypt elevado de 10 a 12
A03	Inyección (Injection)	NO AUDITADO	Mitigación estructural por uso de Prisma ORM (queries parametrizadas) y class-validator; sin auditoría dirigida
A04	Diseño inseguro (Insecure Design)	NO AUDITADO	Fuera de alcance en esta ronda

A05	Configuración insegura (Security Misconfiguration)	NO AUDITADO	Fuera de alcance en esta ronda
A06	Componentes vulnerables (Vulnerable & Outdated Components)	PARCIAL	Auditoría en CI (informativa) + dependencias directas parcheadas (handlebars, nodemailer); deuda transitiva restante sin sanear ACTUALIZADO
A07	Fallas de identificación y autenticación	NO AUDITADO	JWT + Passport ya en uso; mejora de A02 aplica parcialmente; sin auditoría dirigida
A08	Fallas de integridad de software y datos	NO AUDITADO	Fuera de alcance en esta ronda
A09	Fallas de registro y monitoreo (Logging & Monitoring)	CORREGIDO	Bitácora de auditoría append-only (login fallido + acceso denegado) + endpoint de consulta paginado ACTUALIZADO
A10	Server-Side Request Forgery (SSRF)	NO AUDITADO	Fuera de alcance en esta ronda

2. A01 — Control de acceso roto (Broken Access Control)

2.1 Hallazgo principal: el aislamiento multi-tenant no se aplicaba en GraphQL

Severidad: Crítica

MediClick es multi-tenant: cada clínica es un tenant y los datos deben filtrarse por `clinicId`. El filtrado automático se implementa con una extensión de Prisma (`prisma.tenant`) que inyecta `clinicId` en las consultas, alimentada por un `TenantInterceptor` que lee el usuario autenticado del request.

El interceptor obtenía el request con `context.switchToHttp().getRequest()`. Esa llamada es válida en endpoints REST, pero en un *resolver* de GraphQL, `getArgByIndex(0)` retorna el **root del resolver**, no el request HTTP — por lo tanto `clinicId` quedaba `null` y la extensión de Prisma no aplicaba ningún filtro de tenant. Como GraphQL es la única API autenticada del sistema (REST solo expone el webhook público de MercadoPago y endpoints internos de reportes/auditoría con el mismo guard de permisos), el mecanismo de aislamiento no protegía efectivamente ninguna consulta real del producto.

Verificación: se reprodujo el `ExecutionContext` real que NestJS construye para resolvers GraphQL (`args = [root, args, {req}, info]`) y se confirmó en tiempo de ejecución que `switchToHttp().getRequest()` devolvía `undefined` /objeto vacío bajo ese contexto.

Corrección: se reemplazó la obtención del request por un helper existente (`getRequestFromContext`) que distingue el tipo de contexto y usa `GqlExecutionContext.create(context).getContext().req` cuando corresponde a GraphQL.

```
// server/src/shared/interceptors/tenant.interceptor.ts
const req = getRequestFromContext(context) as { user?: { clinicId?: number | null } };
const clinicId: number | null = req?.user?.clinicId ?? null;
```

Mitigación adicional ya existente (defensa en profundidad): las mutaciones de escritura por id ya seguían, en su mayoría, el patrón *read-before-write* (leer el recurso antes de escribir), lo que acotaba el impacto del bug a IDOR de **lectura** en queries — la escritura estaba parcialmente protegida incluso antes del fix.

2.2 Auditoría de read-before-write en escrituras por id

Se revisaron las ~30 mutaciones que escriben por id (`update` / `delete` / `upsert` sobre clave primaria) para verificar que ninguna permita modificar un recurso de otro tenant (IDOR de escritura). Conclusión: **el invariante se cumple** en todos los casos, mediante una de tres vías:

- **Lectura ya filtrada por tenant** (`prisma.tenant.X.findFirst`) — appointments, doctors, medical-history, prescriptions, specialties.
- **Lectura + verificación explícita** de `clinicId` en el caso de uso, lanzando `ForbiddenException` si no coincide — availability, categories, holidays, schedule-blocks, roles, users.
- **Verificación de pertenencia (ownership)** contra el paciente autenticado del token — lista de espera (cancelar / aceptar oferta).

2.3 Hallazgo secundario: updateClinic sin verificación de tenant

Severidad: Baja (riesgo latente, no explotable hoy)

La mutación que actualiza una sede (`UpdateClinicUseCase`) no verificaba que el solicitante solo pudiera modificar su propia sede. No era explotable en el estado actual del sistema porque únicamente el permiso `MANAGE:ALL` (rol de plataforma) tiene `UPDATE:CLINICS` asignado, y para ese rol el acceso cruzado entre sedes es intencional. Aun así, se corrigió como defensa en profundidad.

Por qué se discriminó por rol de sistema y no por clinicId: el usuario "admin de clínica" sembrado en el sistema usa el rol de plataforma ADMIN (equivalente a `MANAGE:ALL`) con un `clinicId` no nulo. Un guard ingenuo del tipo `if (clinicId && id !== clinicId)` habría bloqueado a ese super-administrador al editar una sede distinta a la suya. La solución correcta discrimina por el **rol de sistema** del solicitante (SUPER_ADMIN/ADMIN pueden administrar cualquier sede; el resto de roles, solo la propia).

```
// server/src/modules/clinics/application/use-cases/update-clinic.use-case.ts
const CROSS_TENANT_ROLES = new Set([SystemRole.SUPER_ADMIN, SystemRole.ADMIN]);

if (!CROSS_TENANT_ROLES.has.callerRoleName) && callerClinicId !== id) {
  throw new ForbiddenException('Solo puede modificar su propia sede');
}
```

2.4 Pruebas agregadas

- `tenant.interceptor.spec.ts` — 4 pruebas: reproduce el contexto GraphQL real, confirma que el bug original existía (regresión), confirma el comportamiento correcto en HTTP, y valida `clinicId` nulo para usuarios cross-tenant.
- `update-clinic.use-case.spec.ts` — 4 pruebas: deniega a rol acotado modificar otra sede, permite modificar la propia, permite a rol de plataforma modificar cualquiera, y `NotFound` antes de evaluar el tenant.

3. A02 — Fallas criptográficas (Cryptographic Failures)

El servicio centralizado de hashing de contraseñas (`PasswordService` , único punto de hashing en tiempo de ejecución del sistema) usaba un *cost factor* de `bcrypt` de `10` . Se elevó a `12` , dentro del rango recomendado por OWASP, para dar margen frente al avance de hardware sin impacto perceptible en el tiempo de login.

```
// server/src/modules/auth/infrastructure/services/password.service.ts
private readonly SALT_ROUNDS = 12; // antes: 10
```

Prueba agregada: se verifica que el cost factor real quede codificado en el hash resultante (prefijo `$2b$12$`) y se confirma el round-trip hash/compare. Los scripts de seed de datos de desarrollo (`prisma/seed.ts`), que usan una contraseña de prueba fija, se dejaron fuera de este cambio por no ser superficie de ataque real.

4. A06 — Componentes vulnerables y desactualizados

Se agregó un job `audit` al pipeline de CI (`.github/workflows/ci.yml`) que corre `pnpm audit --prod --audit-level high` sobre ambos workspaces (`server` y `client`) en cada push/PR a `main` .

Por qué quedó en modo informativo (`continue-on-error`): al ejecutar la auditoría se detectaron vulnerabilidades high/critical preexistentes en dependencias de producción (mayormente transitivas, arrastradas por Express, Prisma y Apollo). Activar el gate como bloqueante hoy habría dejado todos los PRs en rojo de forma inmediata. Se prioriza visibilidad continua sobre bloqueo prematuro; el gate debe subirse a obligatorio una vez saneada la deuda.

4.1 Actualización 9 de julio de 2026 — dependencias directas parcheadas

Cerrado (commit `882474d`): se evaluaron las dos dependencias directas señaladas como pendientes en la ronda 1.

- **handlebars** `^4.7.8` → `^4.7.9` — parche, bajo riesgo. Elimina 5 advisories high/critical (inyección JS por AST type confusion vía `@partial-block` , DoS por decorator syntax malformado, inyección en el CLI precompiler).
- **nodemailer** `^8.0.3` → `^9.0.3` — cambio mayor de versión, evaluado y aplicado. El único breaking change de la 9.0.0 es que las peticiones HTTPS para contenido remoto (attachments por URL, endpoints OAuth2, proxy CONNECT) ahora validan el certificado TLS por defecto. `MailService` de MediClick solo usa `createTransport` / `sendMail` / `verify` con SMTP básico, sin attachments remotos ni OAuth2, por lo que el cambio no afecta el comportamiento actual. Se actualizó también `@types/nodemailer` de `^7.0.11` a `^8.0.1` (DefinitelyTyped aún no publica una versión etiquetada para la serie 9.x, pero la superficie de API no cambió).

Resultado medido: `pnpm audit --prod --audit-level high` bajó de 20 a 14 advisories; severidad crítica de 1 a 0; alta de 19 a 14.

Deuda restante (transitiva, no atacada): `hono` / `@hono/node-server` (vía `prisma` / `@prisma/dev`), `multer` (vía `@nestjs/platform-express` y `@nest-lab/throttler-storage-redis`), `ws` (vía

@nestjs/graphql /Apollo), lodash , path-to-regexp , picomatch , defu , effect . Ninguna es dependencia directa del proyecto; resolverlas requiere pnpm.overrides caso por caso, con riesgo real de incompatibilidad con las versiones de NestJS/Prisma/Apollo en uso. Se deja fuera de esta ronda a propósito — el gate de CI permanece informativo hasta evaluarlas una por una.

5. A09 — Fallas de registro y monitoreo de seguridad

El sistema no dejaba rastro de eventos de seguridad relevantes (intentos de login fallidos, accesos denegados por permisos). Se implementó una bitácora de auditoría dedicada:

- **Tabla SecurityAuditLogs** (append-only, sin claves foráneas de forma deliberada: el registro debe sobrevivir el borrado del usuario o la clínica que lo originó; userId/email/clinicId se guardan como snapshot del momento del evento).
- **SecurityAuditService** : registra eventos de forma *fire-and-forget* — un fallo al escribir la auditoría nunca convierte una respuesta 401/403 en un error 500 ni bloquea la petición del usuario.
- **Eventos cubiertos:** LOGIN_FAILED (capturado en AuthController al fallar la autenticación) y PERMISSION_DENIED (capturado en PermissionsGuard al denegar acceso por falta de permisos).

5.1 Actualización 9 de julio de 2026 — endpoint de consulta agregado

Cerrado (commit cc7b24c): la ronda 1 dejó la bitácora en solo-escritura, sin vista de lectura para el equipo de operaciones. Se agregó:

- SecurityAuditService.findAllPaginated() — consulta paginada con filtros opcionales por eventType y rango de fechas (from / to).
- GET /security-audit-logs en un SecurityAuditController nuevo, siguiendo el mismo patrón REST ya usado por el módulo de reportes (@Auth() + @RequirePermissions('READ', 'SECURITY_AUDIT')).
- Subject SECURITY_AUDIT agregado al catálogo de permisos sembrado (prisma/seed.ts) para poder asignarlo a roles granulares en el futuro. No fue necesario para que ADMIN accediera: PermissionsGuard ya trata MANAGE:ALL como wildcard total sobre cualquier acción/subject.

El alcance de los eventos capturados (solo LOGIN_FAILED y PERMISSION_DENIED) se mantiene igual que en la ronda 1, deliberadamente acotado para evitar ruido.

6. Categorías no auditadas en esta ronda

Las siguientes categorías no fueron objeto de revisión dirigida durante este trabajo. Se listan las mitigaciones estructurales ya presentes en el stack del proyecto, sin que constituyan una auditoría formal.

- **A03 — Inyección:** el uso exclusivo de Prisma ORM (consultas parametrizadas) y class-validator en los DTOs del servidor reduce la superficie de inyección SQL/NoSQL clásica, pero no se auditó validación de entrada caso por caso ni inyección en otros vectores (logs, comandos del sistema, plantillas de correo).
- **A04 — Diseño inseguro:** sin revisión de modelado de amenazas formal.

- **A05 — Configuración insegura:** sin auditoría de headers HTTP, CORS, variables de entorno expuestas, ni configuración de Docker/infraestructura. (Nota: `helmet` y CORS restringido a `CLIENT_URL` ya están wireados en `main.ts` por trabajo posterior no relacionado a esta auditoría; no se ha validado formalmente contra A05.)
- **A07 — Fallas de identificación y autenticación:** el sistema ya usa JWT + Passport con rotación de tokens; la mejora de A02 (bcrypt) aporta parcialmente a esta categoría, pero no se revisaron políticas de bloqueo de cuenta, fuerza de contraseña, ni expiración de sesión.
- **A08 — Fallas de integridad de software y datos:** sin revisión de integridad de pipeline de CI/CD ni de deserialización de datos no confiables.
- **A10 — Server-Side Request Forgery (SSRF):** sin auditoría de llamadas salientes del servidor (p.ej. integración con MercadoPago, envío de correo) que reciban URLs o destinos influenciados por el usuario.

7. Commits y trazabilidad

Commit	Categoría	Descripción
<code>e59d67f</code>	A01	Tenant scoping en GraphQL (no solo REST)
<code>cdbcd4b</code>	A09	Bitácora de auditoría de eventos de seguridad
<code>b9a8f95</code>	A02	Cost factor de bcrypt de 10 a 12
<code>247b27f</code>	A06	Auditoría de dependencias en CI
<code>5ecc615</code>	A01	Guard de tenant en updateClinic (defense-in-depth)
<code>90d25c5</code>	—	Limpieza de lint introducido por los cambios de seguridad
<code>e754494</code>	—	Merge a main vía PR #35 (cierre ronda 1)
<code>882474d</code>	A06	Bump handlebars/nodemailer — cierre de pendiente (ronda 2)
<code>cc7b24c</code>	A09	Endpoint de consulta de la bitácora — cierre de pendiente (ronda 2)

8. Verificación

- Suite de pruebas completa del backend (ronda 2): **48 suites / 332 pruebas**, todas en verde tras integrar el bump de dependencias y el endpoint de auditoría.
- `tsc --noEmit` : 0 errores.
- `pnpm audit --prod --audit-level high` (server): 14 advisories restantes, todas transitivas — 0 críticas (antes 1), 14 altas (antes 19).
- Lint limpio (0 errores/warnings nuevos) en los archivos agregados/modificados de esta ronda.
- Migración de base de datos de `SecurityAuditLogs` aplicada y verificada mediante inserción/borrado de prueba en la base de datos real (ronda 1).

9. Hallazgos pendientes y deuda conocida

- **A06:** ~14 vulnerabilidades high transitivas en dependencias de producción (hono, multer, ws, lodash, path-to-regexp, picomatch, defu, effect), sin sanear; requieren `pnpm.overrides` evaluados caso por caso por riesgo de incompatibilidad. El gate de CI permanece informativo hasta resolverlas.
- **Deuda de calidad de código (no es un hallazgo OWASP):** el pipeline de CI tiene el paso de *lint* en rojo de forma crónica (~1740 problemas preexistentes de `@typescript-eslint/no-unsafe-*` en todo el código base, no introducidos por este trabajo). Sanear el resto es un esfuerzo independiente.
- **A03, A04, A05, A07, A08, A10:** pendientes de una ronda de auditoría dirigida.

Resueltos en esta actualización (9 jul 2026): saneamiento de dependencias directas vulnerables de A06 (handlebars, nodemailer) y endpoint de consulta de la bitácora de A09 — ambos figuraban como pendientes puntuales en la versión anterior de este informe.

Informe generado a partir del trabajo de remediación de seguridad realizado sobre la rama `feat/payments-mp-redirect`, integrada a main el 30 de junio de 2026 (PR #35, commit de merge `e754494`). Actualizado el 9 de julio de 2026 con el cierre de los pendientes de A06 y A09 sobre la rama `feat/interop-fase1-proyeccion` (commits `882474d`, `cc7b24c`). Referencia: OWASP Top 10:2021 — owasp.org/Top10.